Final Review

Z. Gu

Fall 2025

Acknowledgement: Lecture slides based on Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C, University of Maine https://web.eece.maine.edu/~zhu/book/

Assembly Programs

http://www.andysinger.com/

Why do we learn Assembly?

- Assembly isn't "just another language".
  - Help you understand how does the processor work
- Assembly program runs faster than high-level language. Performance critical codes must be written in assembly.
  - Use the profiling tools to find the performance bottle and rewrite that code section in assembly
  - Latency-sensitive applications, such as aircraft controller
  - Standard C compilers do not use some operations available on ARM processors, such ROR (Rotate Right) and RRX (Rotate Right Extended).
- Hardware/processor specific code,
  - Processor booting code
  - Device drivers
  - A test-and-set atomic assembly instruction can be used to implement locks and semaphores.
- Cost-sensitive applications
  - Embedded devices, where the size of code is limited, wash machine controller, automobile controllers
- The best applications are written by those who've mastered assembly language or fully understand the low-level implementation of the high-level language statements they're choosing.

Character String

char str[13] = "ARM Assembly";

Big Endian or Little Endian?

Memory Address	Memory Content	Letter
str + 12→	0x00	\0
str + 11→	0x79	y
str + 10→	0x6C	l
str + 9→	0x62	b
str + 8→	0x6D	m
str + 7→	0x65	e
str + 6→	0x73	s
str + 5→	0x73	s
str + 4→	0x41	A
str + 3→	0x20	space
str + 2→	0x4D	M
str + 1→	0x52	R
str →	0x41	A

Data Representation

- Integers
  - Binary in different bases: binary, octal, hex, decimal
  - Signed Integers
    - Two's complement
  - Arithmetic Operations
    - NVCZ flags
  - Big Endian vs Little Endian
- ASCII values
  - Null-terminated string
  - Converting between numbers and ASCII
  - Upper case, lower case
- Fixed point numbers
  - Qm.n, UQm.n
  - Addition, subtract, multiplication, division
- Floating point numbers
  - IEEE 754 Standard
  - Single Precision, Double Precision

Basic Assembly Programming

- Load-modify-store sequence
- Accessing memory
  - Memory addressing mode
    - Pre-index
    - Post-index
    - Pre-index with update
- Data processing
  - Arithmetic, Logic, Comparison, Data Movement
  - Barrel Shifter:
    - ADD r1, r0, r0, LSL 2
  - Bit operations
    - Set a bit, Reset a bit, Toggle a bit, Check a bit
  - LSL, LSR, ASR, ROR, RRX
- Flow control: if, if-then-else, for loop, while loop
  - Unconditional Branch: B
  - Conditional Branch: CMP, TEQ, TST, BEQ, BNE, BMI, BLS, BHI, etc.
  - Conditional Execution: MOVEQ, MOVNE

Subroutine Without Floating Numbers

- Calling Subroutine & Exiting Subroutine
  - BL and BX
  - How are PC, LR, and SP registers updated?
- Caller-saved registers:
  - R0–R3: Arguments/return registers.
  - R12 (IP): Intra-procedure scratch register.
  - CPSR: — caller must preserve its state if needed.
- Callee-saved registers:
  - R4–R11: Must be saved/restored by the callee if used.
  - R14 (LR): Must be saved if the callee makes nested calls.
  - R13 (SP): Stack pointer — must be preserved.
- Extra parameters passed on stack:
  - When more than four arguments exist.
- Return value in R0
- Recursive functions
  - How does the stack work?
  - Pay special attention to **LR** register

Subroutine with Float/Double Arguments

- Each number is assigned in turn to the next free register of the corresponding type

- Example:
  - `double fun(double a1, float a2, double a3, float a4, float a5, double a6, float a7, double a8)`

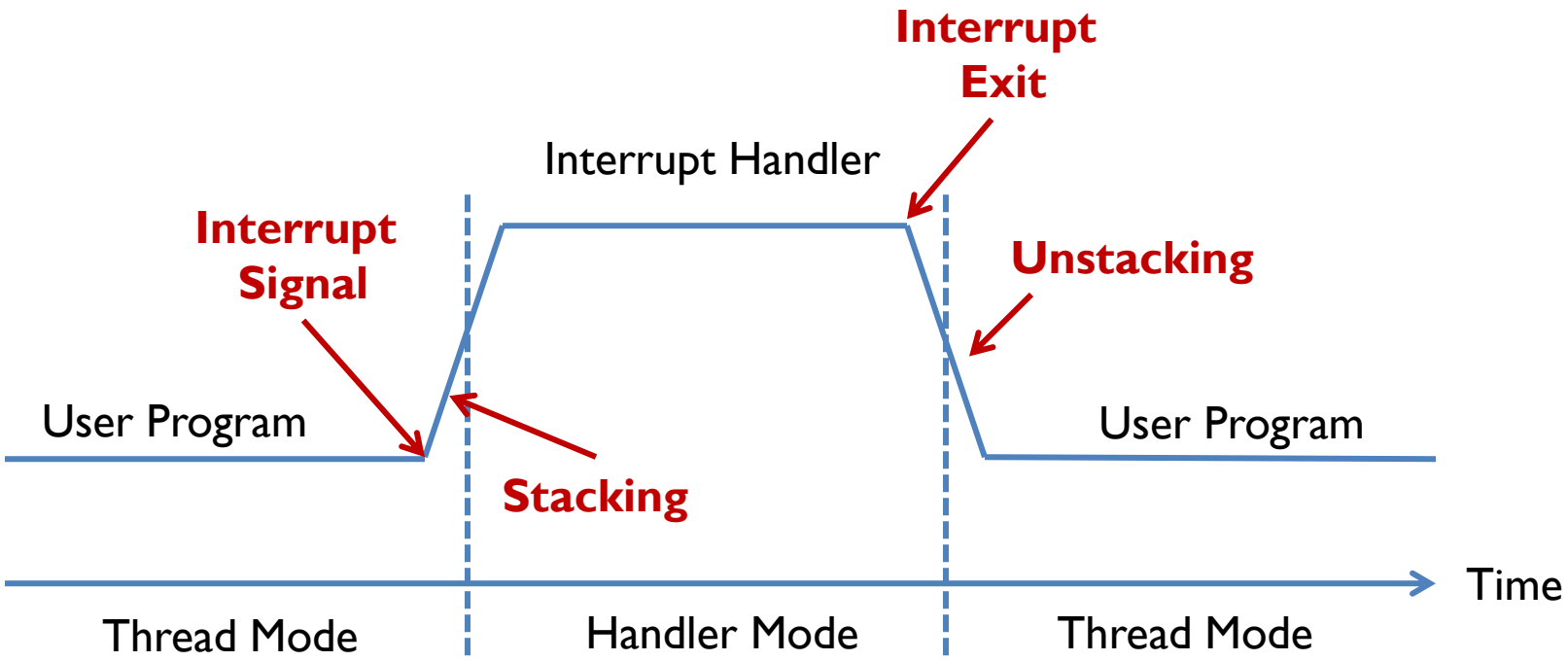
Double-Precision View	D0		D1		D2		D3		D4		D5		D6		D7	
Single-precision View	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15
Argument View	a1		a2	a4	a3		a5	a7	a6		a8					

Interrupt

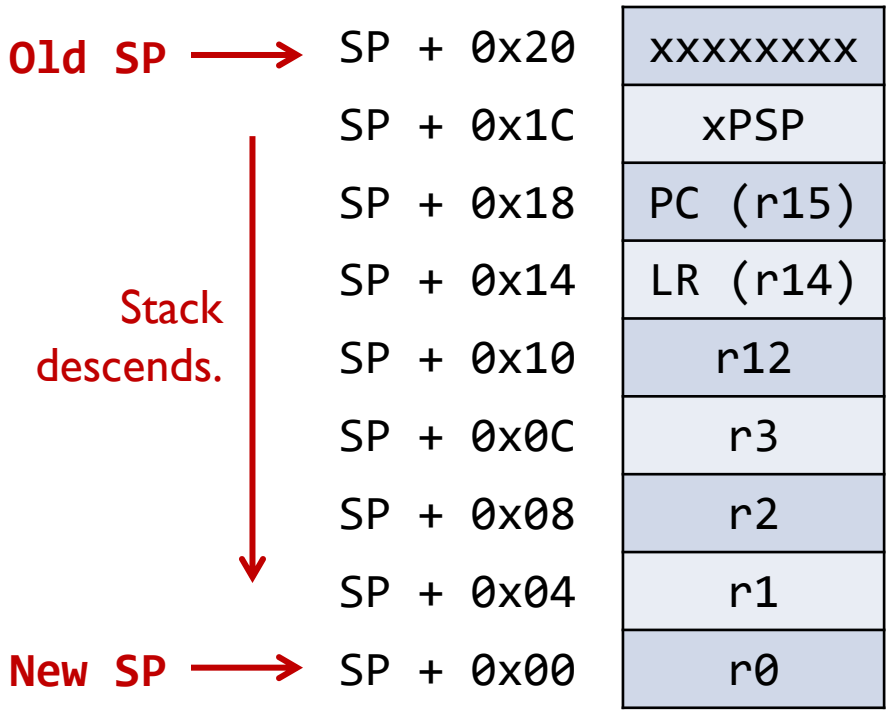
- Interrupt vs Polling
- Interrupt Service Routine
- Interrupt Enable and Disable
- Interrupt Stacking and Unstacking
  - Eight registers (R0-R3, R12, LR, PC, PSR) are automatically pushed into the stack before the hander starts
  - These registers are automatically popped when exiting the handler

http://www.istockphoto.com/

Stacking & Unstacking



Stacking & Unstacking



- The processor automatically pushes these eight registers into the main stack before an interrupt handler starts
- The processor automatically pops these eight register out of the main stack when an interrupt hander exits.

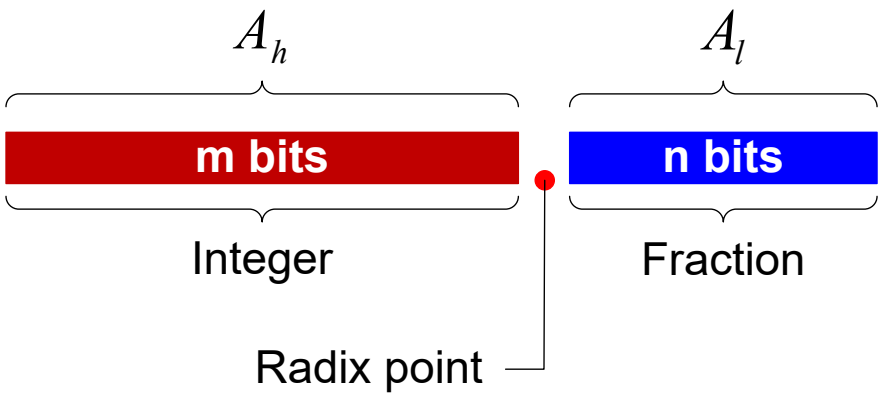
- Two SPs: Main SP (MSP) and Process SP (PSP)
- Determined by operating mode, and CONTROL[0]
  - Thread mode ⟶ SP = PSP
  - Handler mode ⟶ SP = MSP if CONTROL[0] = 0; Otherwise SP = PSP

Mixing C and Assembly

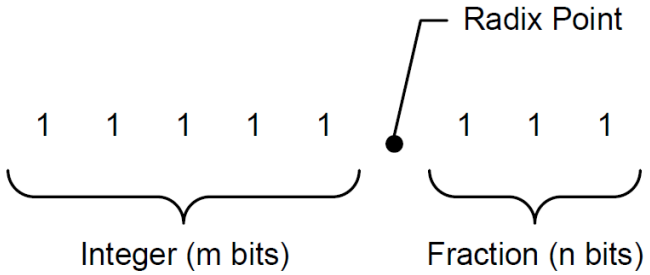
- Accessing C variables
  - Struct & Data alignment
  - Static variables
- Inline Assembly
- C calling assembly functions
  - C: extern assembly functions and variables
  - Assembly: Export
- Assembly calling C functions
  - Assembly: Import C function and variables
  - Export variables to C functions

Unsigned Fixed-point Representation

UQm.n



$$f = A_h + A_l \times 2^{-n}$$



$$10101.101_2 = A_h + A_l \times 2^{-3}$$
$$= 21 + 5 \times 2^{-3}$$
$$= 21.625$$

Signed Fixed-point Representation

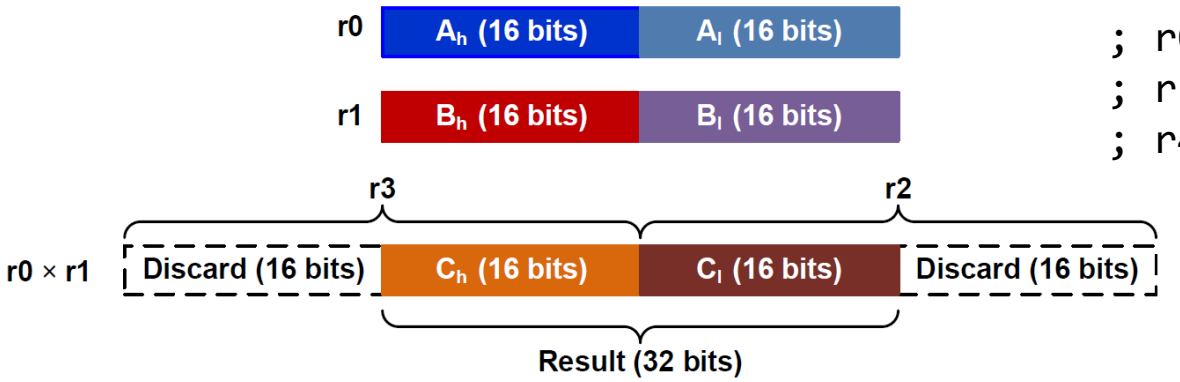
Qm.n

$$A = -1 \times b_{N-1} \times 2^{N-1} + \sum_{i=0}^{N-2} \left(b_i \times 2^i \right)$$

$$f = \frac{A}{2^n}$$

where $N = m + n + 1$

Fixed-point Multiplication in **Q15.16**




```
; r0 = fixed-point number A
; r1 = fixed-point number B
; r4 = fixed-point product = A × B
```

```
SMULL r2, r3, r0, r1    ; Unsigned long multiply
                        ; r2 = low word,  r3 = high word

LSLS  r3, r3, #16       ; Shift left high word

LSRS  r2, r2, #16       ; Shift right low word

ORR   r4, r2, r3        ; Pack two halfwords into a word
```

IEEE Std 754

Single Precision (32 bits)

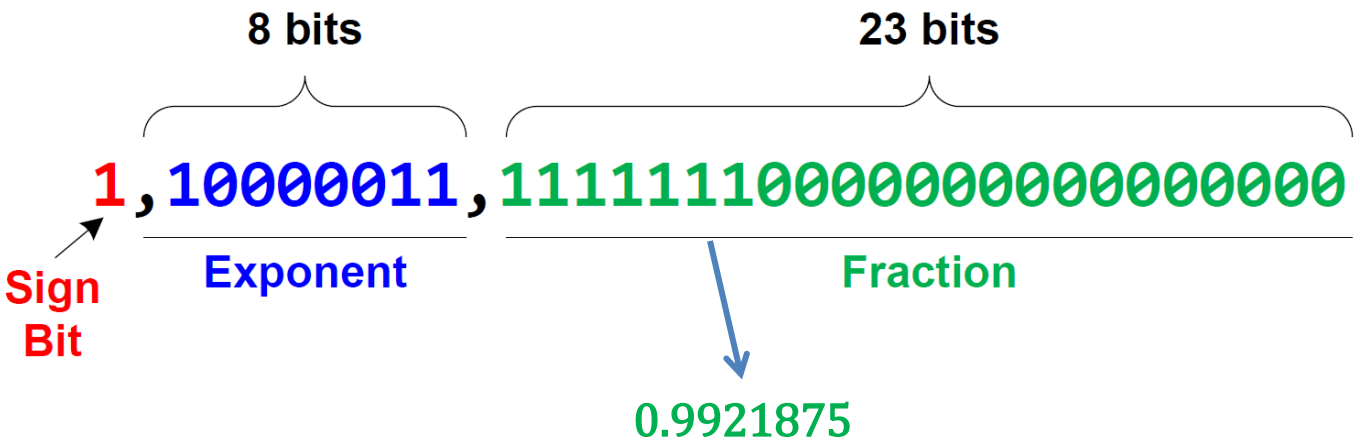
Double Precision (64 bits)

IEEE 754 value:

$$(-1)^{S} \times (1 + Fraction) \times 2^{Exponent - Bias}$$

where Bias = **127** for single precision
Bias = **1023** for double precision

Decoding 0xC1FF0000 into a floating-point number



$$
\begin{aligned}
f &= (-1)^S \times (1 + Fraction) \times 2^{Exponent-127} \\
&= (-1)^1 \times (1 + 0.9921875) \times 2^{131-127} \\
&= -1 \times 1.9921875 \times 2^4 \\
&= -31.875
\end{aligned}
$$

Encoding 14.5 into IEEE Std 754 Single-Precision

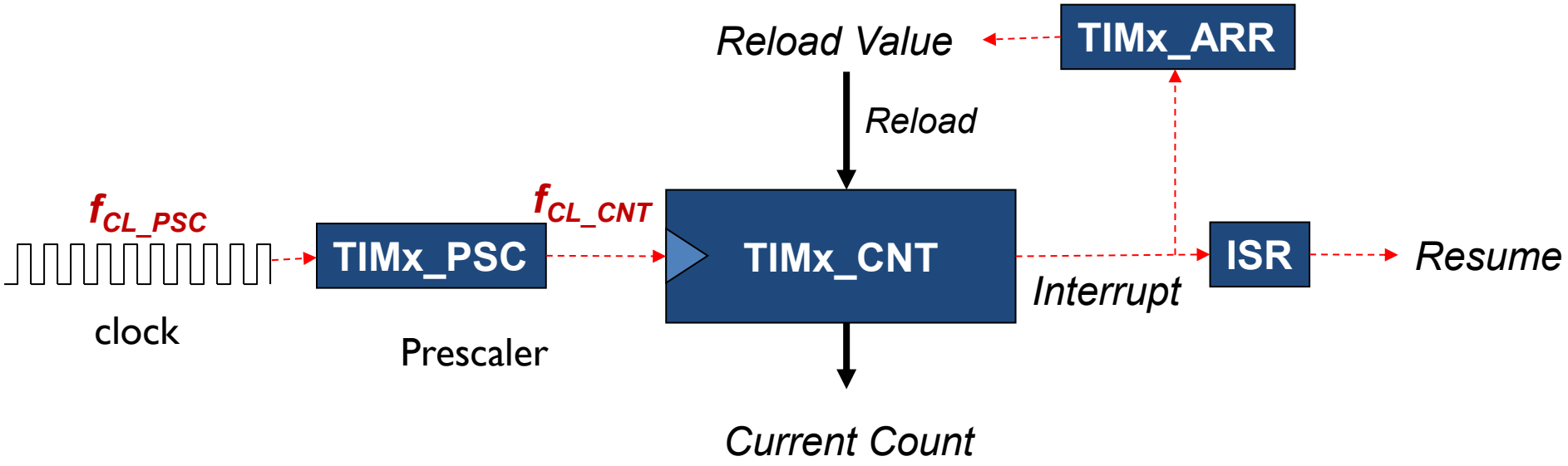
Normalization:

$$14.5 = 1.8125 \times 2^3$$

$$= (1 + 0.8125) \times 2^3$$

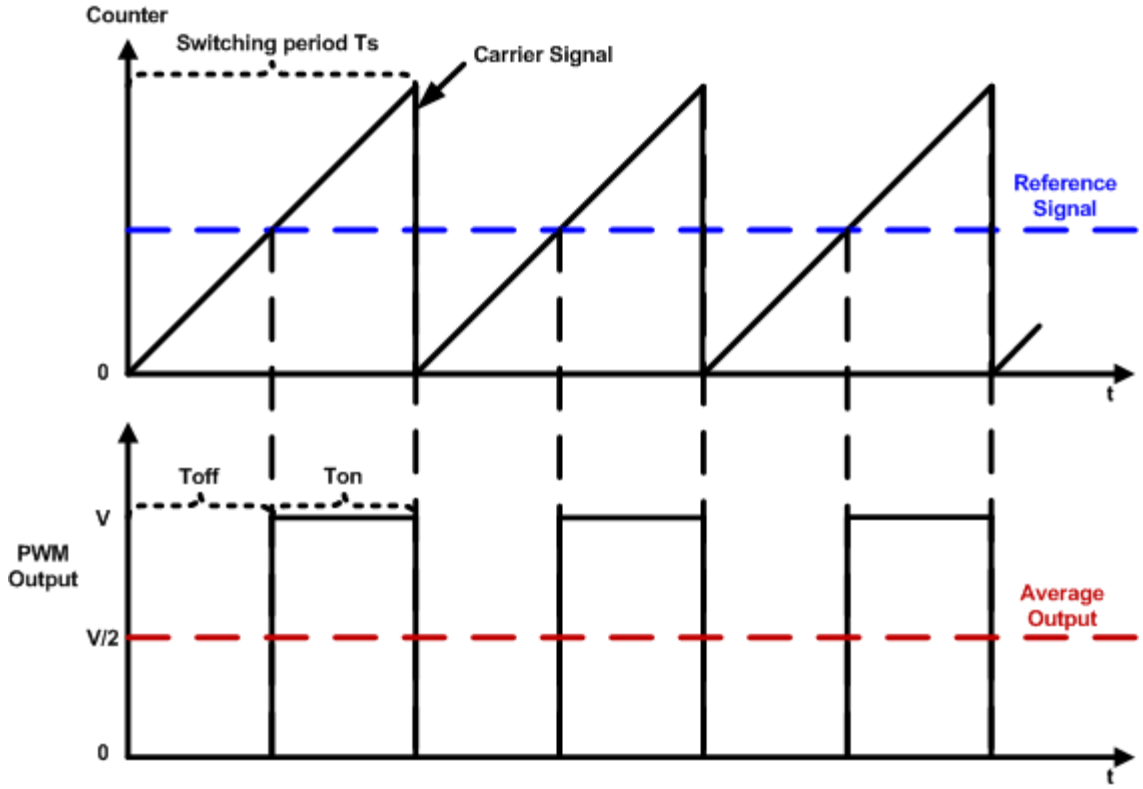
- $Sign = 0$
- $Exponent = 3 + 127 = 130 = 1000010_2$
- $Fraction = 0.1101_2$
  - $0.8125 \times 2 = 1.625 = 1 + 0.625$
  - $0.625 \times 2 = 1.25 = 1 + 0.25$
  - $0.25 \times 2 = 0.5 = 0 + 0.5$
  - $0.5 \times 2 = 1$
- 14.5 = 01000001011010000000000000000000 in binary or 0x41680000 in hex

Timer's Clock



$$f_{CK\_CNT} = \frac{f_{CL\_PSC}}{Prescaler + 1}$$

PWM Duty Cycle = T_{on}/Time Period



$$duty\ cycle = \frac{pulse\ on\ time}{pulse\ switching\ period} \times 100\% = \frac{T_{on}}{T_s} \times 100\%$$